

“FLIGHT SEARCH AND BOOKING SYSTEM”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

SAI SURENDRANATH PENDYALA 2520030157

MUTYALA SAI AJITH REDDY 2520090186

Under the Esteemed Guidance of

Dr Neelima Gogineni

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

**K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **“FLIGHT SEARCH AND BOOKING SYSTEM”** is a record of Bonafide work of **SAI SURENDRANATH PENDYALA - 2520030157, MUTYALA SAI AJITH REDDY - 2520090186** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

**SAI SURENDRANATH PENDYALA – 2520030157
MUTYALA SAI AJITH REDDY - 2520090186**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is certify that the mini project based report entitled “**Design and Development of an Integrated Tourism Platform for Flights, Trains, Hotels, and Hidden Destinations**” is a bonafide work done and submitted by **SAI SURENDRANATH PENDYALA - 2520030157** , **MUTYALA SAI AJITH REDDY - 2520090186** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2025 - 2026** .

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr Madhavi Karumudi**, and my guide **Dr Neelima Gogineni**, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)/Dr. Kaja Shareef (CSIT)**, Head of the *Department for Computer Science Engineering/ Department for Computer Science and Information Technology* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

SAI SURENDRANATH PENDYALA – 2520030157

MUTYALA SAI AJITH REDDY – 2520090186

Abstract

The Flight Search and Booking System is a web-based application developed to simplify the process of searching, comparing, and booking airline tickets through a single platform. The system enables users to search for available flights based on departure city, destination, travel date, number of passengers, and travel class. It provides real-time flight information, fare details, airline names, departure and arrival timings, and seat availability.

The application allows users to create accounts, securely log in, book flights, manage bookings, cancel reservations, and download e-tickets. The system also provides an administrative interface for managing airlines, flights, schedules, bookings, and users. Secure authentication, database management, and responsive web design ensure smooth performance across desktops and mobile devices.

The project aims to improve the overall travel booking experience by offering a fast, secure, and user-friendly platform that minimizes manual effort while maximizing convenience and efficiency.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	5 - 11
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	12 - 14
3	Technology Stack	3.1 Front-end (e.g., React.js, Angular, HTML/CSS) 3.2 Back-end (e.g., Node.js, Django, Spring Boot) 3.3 Database (e.g., MongoDB, MySQL, PostgreSQL) 3.4 Version Control (e.g., Git, GitHub) 3.5 APIs or External services (e.g., Firebase, Stripe)	15 - 20
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer (frontend, backend, database) 4.3 Deployment architecture (e.g., cloud, local, CI/CD pipelines)	21 - 23
5	Design	5.2 ER Diagram / Database schema	24
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic (UI rendering, state management) 6.3 API integration 6.4 Authentication & Authorization	25 - 28
7	Features	7.1 List of main features 7.2 How each feature works (user perspective + technical description)	29 - 33
8	Testing	8.1 Postman testing (frontend/backend) 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	34 - 36
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	37 - 39
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	40 - 42
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	43 - 44
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	45 - 47
13	References	Books used	48

14	Appendices	- Screenshots of the app - Sample code snippets - User manual or guide -Geo Tag photos with guide -Installation / Setup Instructions	49 - 53
----	------------	--	---------

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

With the rapid growth of the aviation industry and digital technology, travelers increasingly prefer online platforms for booking airline tickets instead of visiting travel agencies. Traditional ticket booking methods consume significant time, require manual verification, and often result in booking errors.

The Flight Search and Booking System addresses these challenges by providing an online platform where users can search flights, compare fares, reserve seats, and manage bookings from anywhere. The system integrates modern web technologies and database management systems to provide a reliable, scalable, and efficient booking solution.

The application enhances customer satisfaction by offering instant booking confirmation, secure payment processing, and easy access to travel information.

1.2 Problem Statement

Existing airline booking systems often face several challenges:

- Difficulty in finding suitable flights quickly.
- Manual booking consumes time.
- Limited fare comparison options.
- Poor user interface in some systems.
- Booking errors due to manual processes.
- Lack of centralized booking management.
- Inconvenient ticket cancellation process.

Therefore, there is a need for an efficient online Flight Search and Booking System that simplifies flight searching, booking, payment, and reservation management while ensuring security and reliability.

1.3 Scope of the Project

The scope of the Flight Search and Booking System includes:

- User Registration and Login
- Flight Search
- Flight Filtering
- Fare Comparison
- Seat Availability
- Flight Booking
- Secure Online Payment
- Booking Confirmation
- Ticket Generation
- Booking Cancellation
- User Profile Management
- Admin Dashboard
- Flight Management
- Airline Management
- Passenger Management
- Reports Generation

Future enhancements include:

- AI-based flight recommendations
- Travel insurance
- Hotel booking integration
- Car rental services
- Mobile application
- Loyalty reward programs

1.4 Objectives of the Project

The objectives of this project are:

- Develop an online flight booking platform.
- Provide real-time flight information.
- Simplify ticket reservation.
- Reduce manual booking effort.
- Ensure secure user authentication.
- Improve customer satisfaction.
- Maintain booking history.
- Generate electronic tickets.
- Enable online payment.
- Allow administrators to manage flights efficiently.

1.5 Importance of the Application

The Flight Search and Booking System offers numerous advantages:

- Saves time during ticket booking.
- Reduces paperwork.
- Provides 24×7 service.
- Improves booking accuracy.
- Offers real-time flight information.
- Enhances customer convenience.
- Secure payment processing.
- Easy cancellation and refund management.
- Supports digital transformation in the airline industry.

1.6 Target Users / Audience

The system is designed for:

- Individual Travelers
- Business Travelers
- Families
- Students
- Travel Agencies
- Airline Staff
- System Administrators

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Minimum Requirements

Processor:

- Intel Core i3 or above

RAM:

- 8 GB

Storage:

- 256 GB SSD

Internet:

- Broadband Connection

Display:

- 1366 × 768 Resolution

2.2 Software Requirements

Operating System:

- Windows 10/11
- Ubuntu Linux

Programming Languages:

- HTML5
- CSS3
- JavaScript

Backend:

- Node.js / Express.js

Database:

- MongoDB / MySQL

IDE:

- Visual Studio Code

Browser:

- Google Chrome
- Microsoft Edge
- Mozilla Firefox

Version Control:

- Git
- GitHub

2.3 Development Tools and Frameworks

- Visual Studio Code
- Git
- GitHub
- Postman
- MongoDB Compass
- Figma
- Bootstrap
- React.js
- Express.js

CHAPTER-3 TECHNOLOGY STACK

3.1 Front-End Design

The front-end of the **Flight Search and Booking System** acts as the interface between users and the application. It is responsible for providing an intuitive, responsive, and interactive environment where users can search for flights, compare fares, reserve seats, complete bookings, and manage their travel history. The interface has been designed with simplicity and usability in mind so that users of different age groups can navigate the system without difficulty.

The front-end is developed using **HTML5, CSS3, JavaScript, Bootstrap 5, and React.js**. HTML5 provides the structural framework of the application by defining web page elements such as forms, navigation menus, buttons, tables, and content sections. Semantic HTML elements improve accessibility and search engine optimization while making the code more organized and maintainable.

CSS3 is used to enhance the visual appearance of the website. Modern styling techniques such as Flexbox, CSS Grid, media queries, animations, gradients, shadows, and transitions create a professional and responsive user interface. Bootstrap 5 is incorporated to ensure that the application automatically adjusts its layout for desktops, laptops, tablets, and smartphones. Responsive components such as navigation bars, cards, forms, tables, alerts, and modals improve the overall user experience.

JavaScript provides dynamic functionality within the application. It enables client-side validation, interactive search forms, dynamic flight listings, seat selection, fare calculation, and instant updates without requiring complete page refreshes. AJAX and Fetch API are used to communicate asynchronously with the backend server, allowing users to retrieve flight data in real time.

React.js is utilized to build reusable UI components including the flight search form, booking summary, passenger details form, payment page, and user dashboard. React's component-based architecture makes the application modular, scalable, and easy to maintain. React Router enables smooth navigation between pages without full page reloads, creating a seamless Single Page Application (SPA) experience.

The homepage provides users with options to search flights by entering departure city, destination city, travel date, number of passengers, and travel class. Search results are displayed dynamically with filters such as airline, departure time, price range, duration, and number of stops. The interface also allows users to sort flights based on price, duration, departure time, or airline preference.

Performance optimization techniques such as lazy loading, code splitting, browser caching, image optimization, and component memoization are implemented to reduce page loading time and improve responsiveness. Accessibility features such as proper labels, keyboard navigation, high contrast colors, and ARIA attributes ensure usability for all users.

Overall, the front-end design provides a modern, visually appealing, and responsive interface that enhances user satisfaction while ensuring smooth interaction with the Flight Search and Booking System.

3.2 Back-End Design

The backend serves as the core processing layer of the Flight Search and Booking System. It manages all business logic, user authentication, flight search operations, booking transactions, payment verification, and communication with the database.

The backend is developed using **Node.js** with the **Express.js** framework. Node.js provides a fast, event-driven, non-blocking runtime environment capable of handling multiple user requests simultaneously. Express.js simplifies API development by offering routing, middleware support, and request handling mechanisms.

The backend follows the **Model-View-Controller (MVC)** architecture, separating application logic into models, controllers, and routes. This modular approach improves maintainability, readability, and scalability.

RESTful APIs are developed to perform operations such as:

- User Registration
- User Login
- Flight Search
- Flight Details Retrieval
- Flight Booking
- Seat Reservation
- Booking Cancellation
- Payment Processing
- Booking History
- User Profile Management
- Admin Operations

User authentication is implemented using **JSON Web Tokens (JWT)**. Passwords are encrypted using **bcrypt** before storing them in the database, ensuring maximum security. Middleware functions verify authentication tokens before granting access to protected resources.

The backend validates all incoming requests to prevent SQL Injection, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), and unauthorized access. Express Validator is used to sanitize user inputs and maintain data integrity.

The application communicates with the database through efficient queries and handles errors using centralized error-handling middleware. Booking transactions are processed atomically to prevent duplicate bookings and maintain consistency. If payment fails, the booking request is automatically rolled back to preserve database integrity.

Payment gateway integration can be implemented using **Stripe**, **Razorpay**, or **PayPal** APIs. Once payment is successfully verified, the booking status is updated automatically, and an electronic ticket is generated for the passenger.

Caching mechanisms, connection pooling, and API optimization techniques are employed to improve response times. Logging libraries such as Winston help monitor server activity and facilitate debugging.

Overall, the backend ensures secure, reliable, and efficient processing of all airline reservation operations.

3.3 Database Design

The database forms the backbone of the Flight Search and Booking System by storing all application data securely and efficiently. A relational database management system such as **MySQL** or **PostgreSQL** is used due to its reliability, scalability, and transactional support.

The primary database tables include:

Users

- User ID
- Name
- Email
- Phone Number
- Password
- Role

Flights

- Flight ID
- Airline Name
- Flight Number
- Source
- Destination
- Departure Time
- Arrival Time
- Fare
- Available Seats

Bookings

- Booking ID
- User ID
- Flight ID
- Passenger Name
- Booking Date
- Seat Number
- Booking Status

Payments

- Payment ID
- Booking ID
- Amount
- Payment Method
- Transaction ID
- Payment Status

Airlines

- Airline ID
- Airline Name
- Contact Information

Relationships between tables are established using primary keys and foreign keys to ensure referential integrity. Indexing is applied on frequently searched fields such as departure city, destination city, travel date, and airline name to improve search performance.

The database supports ACID (Atomicity, Consistency, Isolation, Durability) properties to ensure reliable transactions. Backup mechanisms, user privileges, and encrypted storage of sensitive information further enhance security.

Database optimization techniques such as indexing, normalization, query optimization, and connection pooling improve overall system performance.

3.4 Version Control

Version control plays an essential role in managing the source code of the Flight Search and Booking System. The project uses **Git** as the distributed version control system and **GitHub** as the cloud repository for collaboration and backup.

Developers create separate branches for individual modules such as flight search, booking management, payment integration, and user authentication. After development and testing, changes are merged into the main branch through Pull Requests.

Git tracks every modification made to the project, allowing developers to revert to previous versions whenever necessary. GitHub provides issue tracking, project boards, documentation hosting, and continuous integration support.

Standard commit message conventions improve project organization and code readability. Sensitive files such as environment variables and API keys are excluded using the `.gitignore` file.

GitHub Actions can automate testing, building, and deployment processes, reducing manual effort and improving software quality.

Version control ensures collaborative development, efficient code management, and reliable project maintenance throughout the software development lifecycle.

3.5 APIs or External Services

The Flight Search and Booking System integrates several external APIs and cloud services to provide real-time information and enhanced functionality.

Flight Search API

The application integrates airline APIs such as **Amadeus**, **Skyscanner**, or **AviationStack** to retrieve live flight schedules, ticket prices, airline information, and seat availability.

Payment Gateway API

Secure online payments are processed through payment gateways such as:

- Razorpay
- Stripe
- PayPal

These gateways support multiple payment methods including debit cards, credit cards, UPI, net banking, and digital wallets.

Email Service API

Services such as **Nodemailer**, **SendGrid**, or **Mailgun** are used to send booking confirmations, OTPs, password reset links, and electronic tickets to users via email.

SMS Notification API

SMS gateways like **Twilio** or **Fast2SMS** can send booking confirmations, travel reminders, and OTP verification messages directly to passengers.

Authentication Service

JWT-based authentication secures user sessions while bcrypt encrypts passwords before storing them in the database.

Maps API

Google Maps API may be integrated to display airport locations, nearby hotels, and navigation assistance.

Cloud Hosting

The application can be deployed using cloud platforms such as:

- AWS EC2
- Microsoft Azure
- Google Cloud Platform
- Render
- Vercel
- Netlify

Database Hosting

Cloud databases such as **MongoDB Atlas**, **Amazon RDS**, or **PlanetScale** provide scalable and secure data storage.

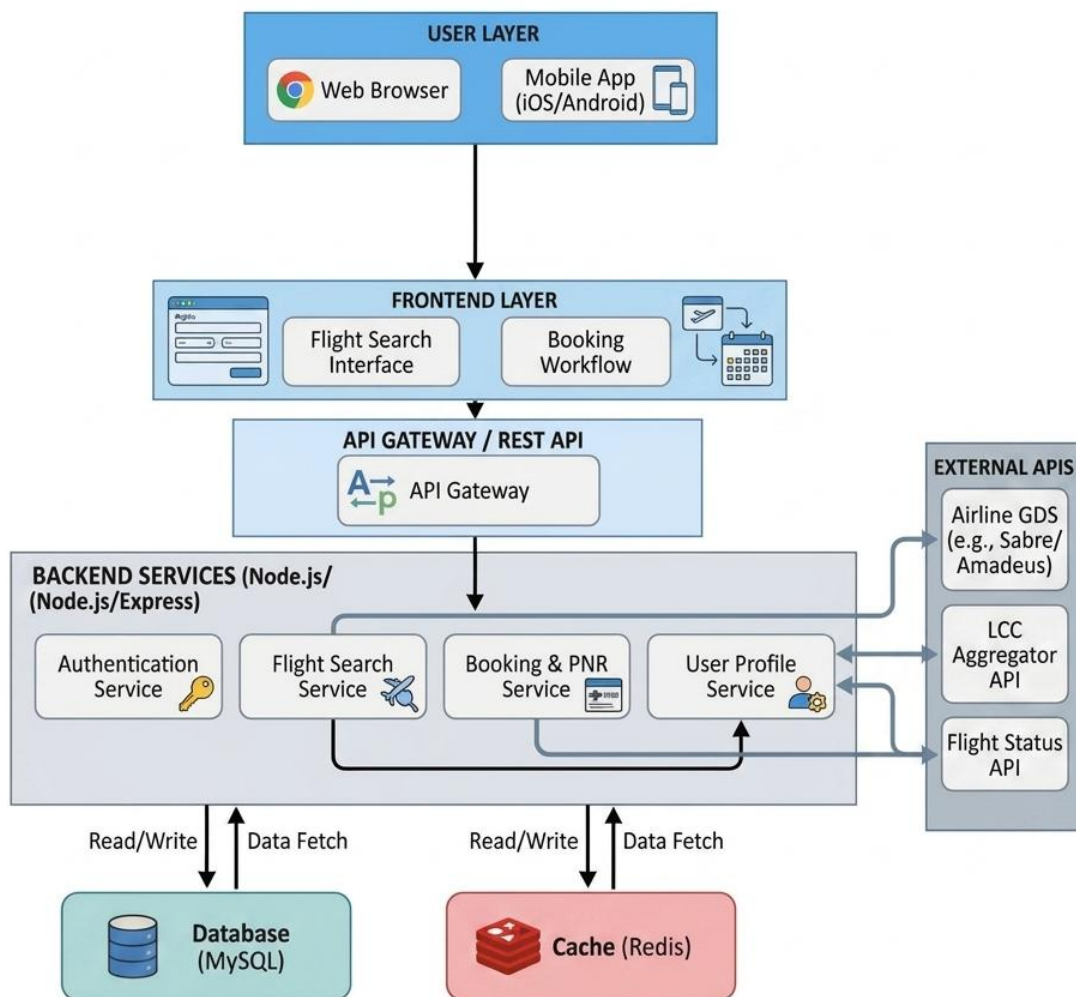
Advantages of API Integration

- Real-time flight information
- Accurate fare comparison
- Secure payment processing
- Automatic ticket generation
- Instant notifications
- Improved customer experience
- Scalability and flexibility
- Reliable cloud-based deployment

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-level architecture diagram

User Architecture Diagram: Flight Search and Booking System



4.2 Description of Each Layer (Frontend, Backend, Database)

The **Flight Search and Booking System** follows a layered architecture that ensures secure, efficient, and reliable communication between users, the application, backend services, databases, and external airline APIs. This architecture improves scalability, maintainability, and overall system performance.

The **User Layer** allows customers to access the system through a web browser or mobile application. Users can register, log in, search for flights, book tickets, manage their profiles, and view booking history using a simple and user-friendly interface.

The **Frontend Layer** is developed using React.js, HTML5, CSS3, JavaScript, and Bootstrap. It provides responsive web pages for flight search, booking, passenger details, and payment. The frontend collects user inputs and sends requests to the backend through REST APIs.

The **API Gateway** acts as an intermediary between the frontend and backend. It receives user requests, validates them, and forwards them to the appropriate backend services while ensuring secure communication.

The **Backend Layer**, built using Node.js and Express.js, contains multiple services such as Authentication Service, Flight Search Service, Booking & PNR Service, and User Profile Service. These services handle user authentication, retrieve real-time flight information, process bookings, generate PNR numbers, manage user accounts, and coordinate payment processing.

The system integrates **External APIs**, including Airline GDS APIs (Amadeus/Sabre), Low-Cost Carrier (LCC) APIs, and Flight Status APIs, to fetch live flight schedules, seat availability, fares, and flight status updates.

To improve performance, the application uses **Redis Cache** for storing frequently accessed data such as popular flight searches and user sessions. This reduces response time and minimizes repeated API requests.

The **MySQL Database** stores all important information, including user accounts, flight details, bookings, payments, and airline data. The backend performs secure read and write operations to maintain data consistency and ensure reliable transaction processing.

Overall, the layered architecture provides a secure, scalable, and efficient solution for the Flight Search and Booking System. It supports real-time flight information, secure booking transactions, high performance, and easy future enhancements, making it suitable for modern airline reservation applications.

4.3 Deployment Architecture (Cloud, Local, CI/CD Pipelines)

The **Flight Search and Booking System** is designed to support both local and cloud-based deployment, ensuring flexibility during development and reliability in production. During the development phase, the application is deployed on a local machine where developers can build, test, and debug the system efficiently. The frontend is developed using **React.js**, while the backend is built using **Node.js** and **Express.js**. The application connects to a **MySQL** database that stores user information, flight details, booking records, and payment data. This local setup allows developers to test new features and resolve issues before deploying the application to a production environment.

For production deployment, the application can be hosted on cloud platforms such as **Amazon Web Services (AWS)**, **Microsoft Azure**, **Google Cloud Platform (GCP)**, **Render**, or **Vercel**. These platforms provide scalable infrastructure, high availability, and reliable performance, allowing the application to handle multiple users simultaneously. The frontend can be deployed on services like **Vercel** or **Netlify**, while the backend can be hosted on cloud servers such as **AWS EC2** or **Render**. The MySQL database can be deployed using managed cloud database services, ensuring secure storage, automatic backups, and easy maintenance.

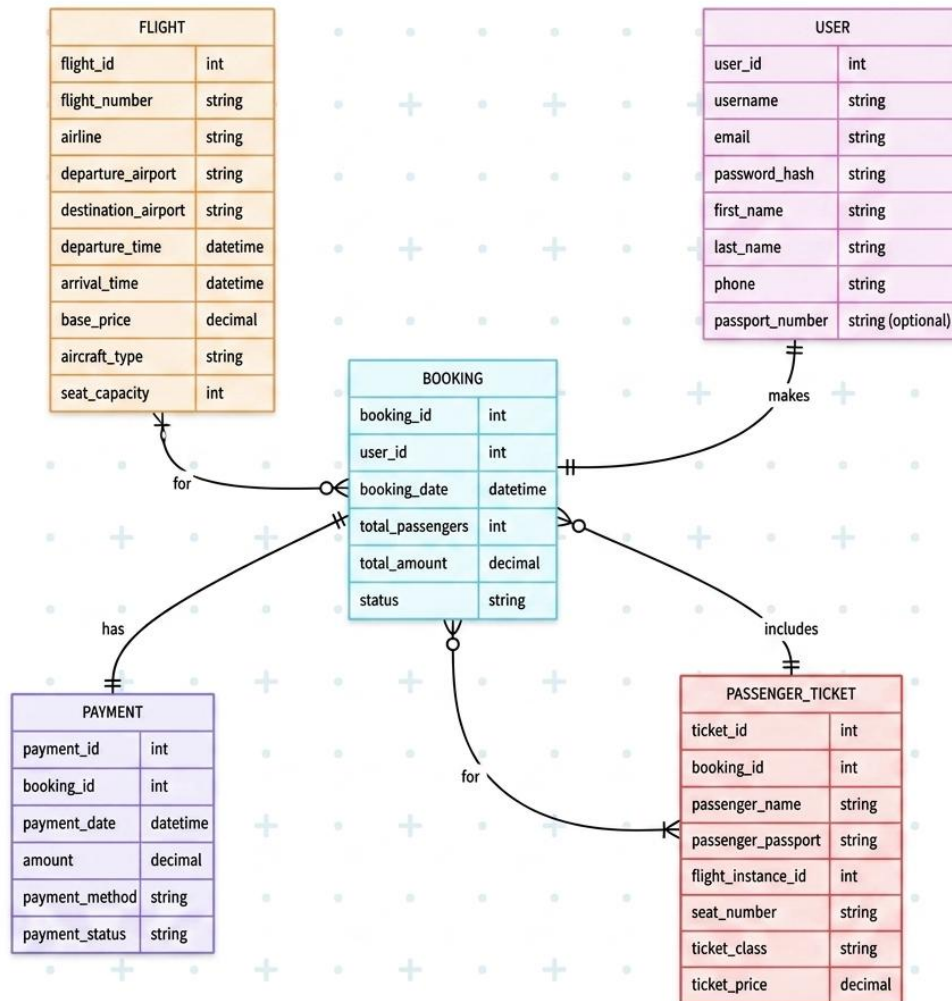
The project uses **Git** for version control and **GitHub** as the remote repository to manage the source code. Git enables developers to track code changes, work collaboratively, and maintain different versions of the application. Team members can develop new features in separate branches and merge them into the main project after proper testing, ensuring better code quality and minimizing conflicts.

To streamline software delivery, the system supports **Continuous Integration (CI)** and **Continuous Deployment (CD)** practices. CI automatically builds and tests the application whenever new code is pushed to the GitHub repository, helping developers identify and fix errors at an early stage. After successful testing, the CD pipeline deploys the updated application to the production server automatically. This automation reduces manual effort, shortens deployment time, minimizes downtime, and ensures that users always access the latest stable version of the application. Tools such as **GitHub Actions**, **Jenkins**, or **GitLab CI/CD** can be used to implement these automated workflows.

Security is also an important aspect of the deployment architecture. The application uses **HTTPS** with **SSL/TLS** encryption to protect data transmitted between users and the server. Sensitive information such as API keys and database credentials is securely stored using environment variables instead of being hardcoded into the source code. Cloud platforms further enhance security through firewalls, access control, regular monitoring, and automated backup services.

Overall, the deployment architecture of the Flight Search and Booking System provides a secure, scalable, and efficient environment for application development and production. By combining local development, cloud hosting, version control, and automated CI/CD pipelines, the system ensures reliable performance, simplified maintenance, faster software delivery, and an improved user experience while remaining adaptable to future enhancements and increasing user demands.

CHAPTER -5 DESIGN



CHAPTER -6 IMPLEMENTATION

Section 6.1 – Module-Wise Implementation

The Flight Search and Booking System is developed using a modular architecture, where each module is responsible for a specific functionality. This approach improves code organization, simplifies maintenance, and allows future enhancements without affecting other components of the application.

The **User Module** manages user registration, login, profile management, and booking history. It allows users to create an account, securely log in, update personal information, and view previously booked flights.

The **Flight Search Module** enables users to search for flights by entering the departure city, destination city, travel date, number of passengers, and travel class. The system processes the search request and displays available flights with details such as airline name, departure time, arrival time, ticket fare, flight duration, and seat availability.

The **Booking Module** handles the reservation process. Once a user selects a flight, the module collects passenger information, verifies seat availability, generates a unique booking ID and PNR number, and stores the booking details in the database.

The **Payment Module** integrates secure online payment gateways such as Razorpay or Stripe. It verifies payment transactions and updates the booking status after successful payment. Upon confirmation, an electronic ticket is generated and sent to the user's registered email address.

The **Admin Module** provides administrative functionalities such as managing flights, updating schedules, monitoring bookings, managing users, and generating reports. Administrators can add, modify, or remove flight information and monitor the overall performance of the system.

Overall, the modular implementation improves system scalability, simplifies debugging, and ensures efficient interaction between different components of the application.

Section 6.2 – Front-End Logic (UI Rendering and State Management)

The front-end of the Flight Search and Booking System is developed using **React.js**, **HTML5**, **CSS3**, **Bootstrap**, and **JavaScript** to provide a responsive and interactive user interface. The application is designed to deliver a smooth user experience across desktops, tablets, and mobile devices.

The user interface dynamically renders different pages such as Home, Flight Search, Search Results, Booking Page, Payment Page, User Dashboard, and Booking History. Based on user interactions, the application updates the displayed information without refreshing the entire page, resulting in faster navigation and improved performance.

State management is responsible for maintaining application data during user interactions. It stores information such as login status, selected flight details, passenger information, booking status, and payment confirmation. Whenever users search for flights or complete a booking, the application automatically updates the interface to reflect the latest information.

Client-side validation is implemented to ensure that users provide valid information before submitting forms. Error messages are displayed for missing or incorrect inputs, improving data accuracy and user experience. The responsive design ensures that all pages adjust automatically to different screen sizes while maintaining consistent functionality.

Overall, the front-end implementation provides a visually appealing, user-friendly, and efficient interface that enables users to search, compare, and book flights with ease.

6.3 API Integration

API integration enables the Flight Search and Booking System to communicate with external services and retrieve real-time information. Instead of storing static flight data, the system connects to airline service providers through RESTful APIs to obtain updated flight schedules, ticket prices, airline information, seat availability, and flight status.

The Flight Search API processes user search requests and returns available flights based on the selected route, travel date, and travel class. Payment Gateway APIs such as Razorpay or Stripe securely process online transactions and confirm payment status before completing the booking process.

The system also integrates Email APIs to send booking confirmations, electronic tickets, and password reset links to registered users. Notification services may be used to send booking alerts and travel reminders.

All API communication follows REST architecture using secure HTTPS connections and JSON data format. Proper validation and error handling ensure reliable communication between the application and external services, even in cases of network delays or API failures.

The use of API integration ensures that users receive accurate, real-time information while improving the overall reliability and functionality of the Flight Search and Booking System.

6.4 Authentication and Authorisation

Authentication and authorization are essential security mechanisms implemented in the Flight Search and Booking System to protect user information and restrict unauthorized access.

Authentication verifies the identity of users during the login process. Registered users enter their email address and password, which are validated against the stored records in the database. Passwords are securely encrypted using **bcrypt**, ensuring that sensitive information remains protected. After successful authentication, the system generates a **JSON Web Token (JWT)** that is used to maintain the user's session during subsequent requests.

Authorization determines what actions each user is permitted to perform after logging into the system. Normal users are allowed to search flights, book tickets, make payments, cancel bookings, and manage their personal profiles. Administrative users have additional privileges, including managing flight schedules, updating airline information, monitoring bookings, managing users, and generating reports.

The system also implements role-based access control, ensuring that users can access only the resources and functionalities assigned to their roles. Protected API endpoints require valid authentication tokens before granting access, preventing unauthorized operations and enhancing system security.

Additional security measures such as HTTPS encryption, input validation, secure session management, and environment variables for storing API keys further strengthen the protection of sensitive user data. These security mechanisms ensure that the Flight Search and Booking System remains reliable, secure, and compliant with modern web application security practices.

CHAPTER 7- FEATURES

7.1 List of Main Features

The Flight Search and Booking System provides a comprehensive set of features that simplify the process of searching, booking, and managing airline tickets. The major features of the system are as follows:

- User Registration and Login
- Flight Search
- Flight Filtering and Sorting
- Flight Booking
- Passenger Information Management
- Secure Online Payment
- Booking Confirmation and E-Ticket Generation
- Booking History
- Booking Cancellation
- User Profile Management
- Admin Dashboard
- Flight Management
- Airline Management
- User Management
- Real-Time Flight Information
- Responsive User Interface
- Secure Authentication and Authorization

These features work together to provide users with a convenient, secure, and efficient online flight booking experience while enabling administrators to manage flights and bookings effectively.

7.2 How Each Feature Works

1. User Registration and Login

From the user's perspective, new users can create an account by entering their personal details such as name, email address, phone number, and password. Registered users can log in securely using their credentials to access the booking services.

Technically, the system validates user inputs, encrypts passwords using **bcrypt**, stores user information in the **MySQL database**, and generates a **JSON Web Token (JWT)** after successful authentication. This token is used to authorize future requests securely.

2. Flight Search

Users can search for available flights by entering the departure city, destination city, travel date, number of passengers, and travel class. The system displays matching flights along with airline details, departure and arrival times, ticket fares, and seat availability.

From a technical perspective, the frontend sends search requests through REST APIs to the backend, which retrieves flight information from the database or external airline APIs and returns the matching results to the user.

3. Flight Filtering and Sorting

To simplify flight selection, users can filter search results based on airline, price, departure time, travel duration, and number of stops. They can also sort flights according to fare, departure time, or duration.

The backend processes these filter and sorting conditions before returning optimized search results, improving the overall user experience.

4. Flight Booking

After selecting a preferred flight, users enter passenger details and confirm the booking. The system reserves available seats and generates a booking confirmation.

Technically, the Booking Service verifies seat availability, creates a booking record, generates a unique Booking ID and PNR number, and stores the information securely in the database.

5. Passenger Information Management

Users provide passenger details such as name, age, gender, contact information, and identification details during the booking process. These details are stored securely and can be reused for future bookings.

The backend validates all passenger information before storing it in the database to ensure data accuracy and consistency.

6. Secure Online Payment

Users can complete their booking by making payments through secure online payment gateways using credit cards, debit cards, UPI, net banking, or digital wallets.

The application integrates payment gateways such as **Razorpay** or **Stripe** to process transactions securely. After successful payment verification, the booking status is updated automatically.

7. Booking Confirmation and E-Ticket Generation

Once the payment is successful, users receive instant booking confirmation along with an electronic ticket containing the booking reference, PNR number, passenger details, and flight information.

The backend generates the booking confirmation and sends the e-ticket to the user's registered email address using an email service.

8. Booking History

Registered users can access their booking history to view previously booked flights, upcoming trips, and cancelled reservations.

The system retrieves booking records associated with the logged-in user from the database and displays them in the user dashboard.

9. Booking Cancellation

Users can cancel eligible flight bookings directly through their account. After cancellation, the booking status is updated, and refund processing is initiated according to airline policies.

The backend validates cancellation requests and updates both the booking and payment records accordingly.

10. User Profile Management

Users can update their personal information, change passwords, and manage account settings through the profile section.

All updates are validated before being securely stored in the database to maintain accurate user records.

11. Admin Dashboard

Administrators access a dedicated dashboard to monitor users, bookings, flights, and system activities. The dashboard provides complete control over the application's operations.

Only authorized administrators can access these features through role-based authentication.

12. Flight Management

Administrators can add new flights, update flight schedules, modify fares, manage seat availability, and remove outdated flight information.

These operations are performed through secure administrative interfaces connected directly to the database.

13. Airline Management

The system allows administrators to manage airline information, including airline names, flight schedules, routes, and operational details.

This ensures that flight information remains accurate and up to date.

14. User Management

Administrators can view registered users, monitor user activity, and manage customer accounts whenever necessary.

This feature helps maintain system security and improves administrative control.

15. Real-Time Flight Information

The application retrieves live flight schedules, ticket fares, seat availability, and flight status from external airline APIs.

This ensures that users always receive accurate and up-to-date flight information before making reservations.

16. Responsive User Interface

The application is designed with a responsive interface that adapts automatically to desktops, laptops, tablets, and smartphones, ensuring a consistent user experience across all devices.

17. Secure Authentication and Authorization

The system protects user accounts through secure login mechanisms, encrypted passwords, JWT-based authentication, and role-based authorization.

These security measures prevent unauthorized access and ensure that users can only access resources according to their assigned roles.

CHAPTER -8 TESTING

8.1 Postman Testing (Frontend/Backend)

Postman was used to test the RESTful APIs developed for the Flight Search and Booking System. It helped verify that the backend services responded correctly to requests sent from the frontend. Each API endpoint was tested independently to ensure proper communication between the client application and the server.

The APIs tested included user registration, login, flight search, flight booking, payment processing, booking history, profile management, and booking cancellation. Different request methods such as **GET**, **POST**, **PUT**, and **DELETE** were used to validate API functionality. Request parameters, headers, authentication tokens, and JSON responses were verified to ensure accurate data exchange between the frontend and backend.

Frontend testing involved verifying that user actions such as searching for flights, booking tickets, and viewing booking history correctly triggered the corresponding backend APIs. Proper error messages were displayed for invalid inputs, and successful API responses were reflected immediately in the user interface. Postman testing confirmed that all APIs functioned correctly and returned appropriate responses under different testing conditions.

8.2 Integration Testing

Integration testing was performed to verify that all modules of the Flight Search and Booking System worked together without any issues. After individual modules were tested successfully, they were integrated to ensure smooth communication between the frontend, backend, database, and external services.

The testing process verified the complete workflow of the application, including user registration, login, flight search, booking, payment processing, booking confirmation, and booking history. Database operations were validated to ensure that user information, flight details, bookings, and payment records were stored and retrieved correctly. Integration with external flight APIs and payment gateways was also tested to ensure that real-time flight information and secure payment processing functioned properly.

The results confirmed that all integrated components communicated efficiently and the system successfully handled user requests from start to finish without data inconsistencies.

8.3 Tools Used for Testing

Several testing tools were used to evaluate the functionality, performance, and reliability of the Flight Search and Booking System.

- **Postman** was used for testing REST APIs by sending HTTP requests and validating API responses.
- **Google Chrome Developer Tools** were used to inspect frontend components, debug JavaScript, monitor network requests, and analyze application performance.
- **Visual Studio Code** provided debugging support during application development.
- **Git and GitHub** were used to manage source code versions and track changes throughout the development process.
- **MySQL Workbench** (or MongoDB Compass, depending on the database used) was utilized to verify database records, execute queries, and monitor stored data.
- **React Developer Tools** helped inspect React components and monitor application state during frontend testing.

These tools simplified debugging, improved code quality, and ensured that the application met the required functional and performance standards.

8.4 Test Cases and Results

The following test cases were executed to verify the functionality of the Flight Search and Booking System.

Test Case ID	Test Scenario	Expected Result	Actual Result	Status
TC-01	User Registration	User account created successfully	Account created successfully	Passed
TC-02	User Login	User logs in with valid credentials	Login successful	Passed
TC-03	Invalid Login	Display error message	Error message displayed	Passed
TC-04	Flight Search	Display available flights	Flights displayed correctly	Passed
TC-05	Flight Booking	Booking completed successfully	Booking confirmed with PNR	Passed
TC-06	Payment Processing	Payment completed successfully	Payment successful	Passed
TC-07	Booking History	Display previous bookings	Booking history displayed	Passed
TC-08	Booking Cancellation	Booking cancelled successfully	Booking cancelled successfully	Passed
TC-09	Profile Update	User profile updated	Profile updated successfully	Passed
TC-10	Admin Flight Management	Flight details updated	Flight updated successfully	Passed

The testing results indicate that all major functionalities of the Flight Search and Booking System operated as expected. The application successfully handled user authentication, flight searching, booking, payment processing, profile management, and administrative operations without significant errors. The successful completion of these test cases demonstrates that the system is reliable, secure, and ready for deployment in a production environment.

CHAPTER -9 DEPLOYMENT

9.1 Steps to Deploy

The deployment of the **Flight Search and Booking System** involves preparing the application for production and making it accessible to users through a web server or cloud platform. Before deployment, all modules of the application are thoroughly tested to ensure that they function correctly without errors.

The first step is to prepare the source code by organizing the frontend and backend into separate directories and installing all required dependencies using package managers such as **npm**. Configuration files are updated with the appropriate environment variables, including database credentials, API keys, and authentication secrets.

Next, the frontend application is built into optimized production files using the React build process. Similarly, the backend server is configured with the required middleware, routes, and database connections. The application is then connected to the production database, where all necessary tables and initial data are created.

After successful configuration, both the frontend and backend are deployed to their respective hosting platforms. The deployed application is tested again to verify user registration, login, flight search, booking, payment processing, and other core functionalities. Finally, the application is monitored regularly to ensure stable performance, timely updates, and secure operation.

9.2 Environment Configuration

Proper environment configuration is essential for the successful deployment of the Flight Search and Booking System. The application requires separate configurations for development and production environments to ensure security, performance, and maintainability.

The backend environment stores sensitive information such as database connection details, JWT secret keys, API keys, and payment gateway credentials using environment variables instead of hardcoding them into the source code. This approach improves security and allows easy configuration across different deployment environments.

The application uses **Node.js** as the runtime environment, **Express.js** for backend services, and **MySQL** as the database management system. The frontend is developed using **React.js**, while communication between the frontend and backend is established through RESTful APIs over secure HTTPS connections.

Version control is managed using **Git** and **GitHub**, allowing developers to maintain different versions of the project and collaborate efficiently. During deployment, the production environment is configured with the required software dependencies, database connections, SSL certificates, and domain settings to ensure secure and reliable operation.

Proper environment configuration also includes enabling error logging, backup mechanisms, and monitoring tools that help maintain application stability and quickly identify any operational issues after deployment.

9.3 Hosting of Frontend and Backend

The Flight Search and Booking System can be hosted on modern cloud platforms that provide scalability, reliability, and high availability. The **frontend**, developed using React.js, can be deployed on platforms such as **Vercel**, **Netlify**, or **AWS S3**, which are optimized for hosting static web applications. These platforms provide fast loading speeds, automatic deployments from GitHub repositories, and secure HTTPS support.

The **backend**, developed using Node.js and Express.js, can be hosted on cloud platforms such as **Render**, **Amazon Web Services (AWS EC2)**, **Microsoft Azure**, or **Google Cloud Platform (GCP)**. These services provide the computing resources required to process API requests, manage user authentication, handle flight bookings, and communicate with the database.

The **MySQL database** can be hosted using managed database services such as **Amazon RDS**, **Azure Database for MySQL**, or **Google Cloud SQL**, which offer secure storage, automatic backups, and high availability. Hosting the database separately from the application server improves scalability and allows independent management of application and data resources.

After deployment, the frontend communicates with the backend through secure REST APIs, while the backend interacts with the database and external flight APIs to process user requests. Cloud hosting ensures that the application remains available to users at all times, supports increasing user traffic through automatic scaling, and provides monitoring, backup, and recovery services to maintain reliable system performance.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

During the development of the **Flight Search and Booking System**, several technical and implementation challenges were encountered. One of the major challenges was integrating the frontend with the backend while ensuring smooth communication through RESTful APIs. Initially, there were issues with API request handling and displaying real-time flight information correctly on the user interface.

Another challenge was implementing secure user authentication and session management. Protecting user credentials, managing login sessions, and restricting unauthorized access required careful implementation of authentication and authorization mechanisms.

Database management was another important challenge. Designing relationships between users, flights, bookings, and payments while maintaining data consistency required proper database normalization and query optimization. Ensuring that booking records and seat availability remained synchronized during multiple booking requests was also a critical task.

Integrating external flight APIs and payment gateways presented additional challenges due to differences in API responses, authentication methods, and occasional network delays. Proper error handling and validation were required to ensure that the application continued functioning even when external services were temporarily unavailable.

Another issue involved developing a responsive user interface that functioned consistently across different browsers and screen sizes. Ensuring compatibility with desktop and mobile devices required additional testing and UI improvements.

10.2 Solutions Applied

Several solutions were implemented to overcome the challenges encountered during development. RESTful APIs were designed with proper request validation and standardized JSON responses to improve communication between the frontend and backend. Error-handling mechanisms were added to display meaningful messages whenever an API request failed.

To strengthen security, user authentication was implemented using **JSON Web Tokens (JWT)**, while passwords were encrypted using **bcrypt** before being stored in the database. Role-based authorization was introduced to ensure that only authorized users could access administrative features.

The database was carefully designed using relational tables with primary and foreign key relationships to maintain data integrity. Query optimization and indexing techniques were applied to improve search performance and reduce response time.

For external API integration, exception handling and fallback mechanisms were implemented to manage network failures and invalid responses. Payment processing was integrated with secure payment gateways to ensure reliable transaction handling.

To improve the user experience, Bootstrap and responsive design techniques were used to create interfaces that adapt automatically to different screen sizes. Cross-browser testing was performed to ensure compatibility across major web browsers.

Version control using **Git** and **GitHub** helped manage code changes efficiently and enabled collaborative development without affecting the stability of the application.

10.3 Current Limitations or Known Bugs

Although the Flight Search and Booking System performs its core functionalities successfully, a few limitations still exist. The application depends on external flight APIs to retrieve real-time flight schedules, fares, and seat availability. If these services become unavailable or experience delays, some flight information may not be updated immediately.

The current version focuses only on flight booking and does not include additional travel services such as hotel reservations, car rentals, travel insurance, or holiday packages. The payment module currently supports selected payment gateways and can be expanded to include additional payment methods in future versions.

Another limitation is that the system currently supports only basic flight search and booking features. Advanced capabilities such as AI-based flight recommendations, multilingual support, offline booking, and personalized travel suggestions are not included in the current implementation.

Despite these limitations, the Flight Search and Booking System successfully meets its primary objectives by providing secure user authentication, real-time flight search, efficient booking management, secure payment processing, and a responsive user interface. The modular architecture allows these limitations to be addressed through future enhancements without requiring major changes to the existing system.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned Features

The Flight Search and Booking System has been designed with a scalable architecture that allows new features to be added in future versions. Although the current system provides essential functionalities such as flight search, booking, payment processing, and user management, several advanced features can further improve the overall user experience and system capabilities.

One of the planned enhancements is the implementation of **AI-based flight recommendations**, where the system suggests suitable flights based on a user's travel history, preferences, and budget. This feature will help users find the best travel options more efficiently.

Another planned feature is the integration of **real-time flight tracking**, allowing users to monitor flight status, departure and arrival times, gate changes, delays, and cancellations directly through the application. Push notifications can also be introduced to provide instant updates regarding flight schedules and booking confirmations.

The system can also be expanded to support **multi-language functionality**, enabling users from different regions to access the application in their preferred language. Additional payment methods such as digital wallets, international payment gateways, and Buy Now Pay Later (BNPL) options can also be integrated to provide greater flexibility.

Future versions may also include **hotel booking, car rental services, travel insurance, and holiday package reservations**, transforming the application into a complete travel management platform. A dedicated mobile application for Android and iOS can further improve accessibility and convenience for users.

11.2 Possible Integrations or Optimisations

Several integrations and optimizations can be implemented to improve the performance, scalability, and functionality of the Flight Search and Booking System. Integration with **Global Distribution Systems (GDS)** such as **Amadeus**, **Sabre**, and **Travelport** can provide access to a larger network of airlines and more accurate real-time flight information. Additional airline APIs can also be integrated to improve flight availability and fare comparison.

Performance can be optimized by implementing advanced caching techniques using **Redis**, reducing API response times and minimizing repeated database queries. Database optimization through indexing, query optimization, and connection pooling can further improve system efficiency, especially when handling a large number of users simultaneously.

Security can be enhanced by introducing **Multi-Factor Authentication (MFA)**, biometric authentication, and advanced encryption techniques to provide additional protection for user accounts and sensitive data. Continuous monitoring, automated backups, and cloud-based disaster recovery solutions can improve system reliability and ensure uninterrupted service.

The deployment process can also be optimized using **Docker containers**, **Kubernetes**, and automated **CI/CD pipelines**, allowing faster deployment, easier scaling, and simplified maintenance. Integration with analytics tools can provide insights into user behavior, booking trends, and system performance, helping administrators make informed decisions.

Overall, these planned features and optimizations will enhance the functionality, security, and scalability of the Flight Search and Booking System, enabling it to support future technological advancements and meet the evolving needs of users in the airline reservation industry.

CHAPTER -12 CONCLUSION

12.1 Summary of the Project

The **Flight Search and Booking System** was developed to provide a secure, efficient, and user-friendly platform for searching and booking airline tickets online. The system simplifies the entire booking process by allowing users to register, log in, search for available flights, compare ticket prices, select suitable flights, make secure online payments, and receive instant booking confirmations. The application also provides features such as booking history, user profile management, and administrative tools for managing flights, users, and bookings.

The project was developed using modern web technologies including **React.js** for the frontend, **Node.js** and **Express.js** for the backend, and **MySQL** for database management. RESTful APIs were used to enable communication between different system components, while secure authentication using **JWT** and password encryption with **bcrypt** ensured data protection. Overall, the project demonstrates the successful implementation of a complete web-based airline reservation system.

12.2 What Was Achieved

The main objective of the project was to develop an online platform that simplifies flight searching and booking while ensuring security, reliability, and ease of use. This objective was successfully achieved by implementing all the core functionalities required for an airline reservation system.

The developed application allows users to search for flights using various criteria, view real-time flight information, complete secure bookings, and manage their reservations efficiently. Secure authentication and role-based authorization were implemented to protect user accounts and administrative operations. The integration of external APIs and payment gateways enabled real-time flight information and secure online transactions.

The project also achieved a responsive and user-friendly interface that works across multiple devices and browsers. The modular architecture, proper database design, and cloud-ready deployment approach make the system scalable and suitable for future enhancements.

12.3 Skills Learned During Development

The development of the Flight Search and Booking System provided valuable practical experience in modern web application development and software engineering. During the project, knowledge of frontend development was strengthened through the use of **React.js**, **HTML5**, **CSS3**, **Bootstrap**, and **JavaScript** to create responsive and interactive user interfaces.

On the backend, practical experience was gained in developing RESTful APIs using **Node.js** and **Express.js**, implementing secure authentication using **JWT**, encrypting passwords with **bcrypt**, and managing application logic. Database design and management skills were improved by working with **MySQL**, including table creation, relationships, queries, and data validation.

The project also provided experience in integrating third-party APIs, implementing secure payment processing, using **Git** and **GitHub** for version control, and deploying applications to cloud platforms. Additionally, important software engineering skills such as debugging, testing, teamwork, problem-solving, documentation, and project management were developed throughout the implementation process.

Overall, this project enhanced both technical knowledge and practical development skills while providing a deeper understanding of full-stack web application development and real-world software deployment.

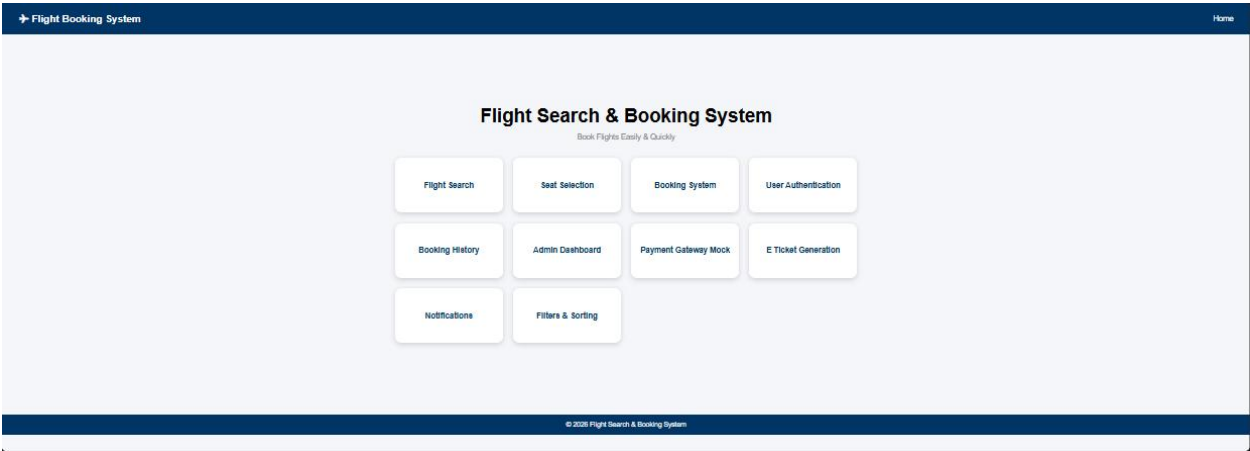
CHAPTER -13 REFERENCES

Books

- [1] M. Nixon, **Learning React: Modern Patterns for Developing React Apps**, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2023.
- [2] A. Banks and E. Porcello, **Learning React**, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2020.
- [3] A. Holovaty and J. Kaplan-Moss, **The Definitive Guide to Web Development**, Apress, 2021.
- [4] D. Flanagan, **JavaScript: The Definitive Guide**, 7th ed. Sebastopol, CA, USA: O'Reilly Media, 2020.
- [5] A. Watt, **Beginning HTML5 and CSS3**, 2nd ed. Birmingham, UK: Packt Publishing, 2018.
- [6] M. Welling and L. Thomson, **PHP and MySQL Web Development**, 5th ed. Boston, MA, USA: Addison-Wesley, 2017. (*Useful for database concepts even if your backend uses Node.js.*)
- [7] K. Samani, **Node.js Design Patterns**, 4th ed. Birmingham, UK: Packt Publishing, 2023.
- [8] E. Robson and E. Freeman, **Head First Design Patterns**, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2020.
- [9] I. Sommerville, **Software Engineering**, 10th ed. Boston, MA, USA: Pearson, 2016.
- [10] A. Silberschatz, H. F. Korth, and S. Sudarshan, **Database System Concepts**, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

CHAPTER -14 APPENDICES

Screenshots of the app



Sample code snippets

```
1  import { Routes, Route } from "react-router-dom";
2
3  import Home from "../pages/Home";
4  import FlightSearch from "../pages/FlightSearch";
5  import SeatSelection from "../pages/SeatSelection";
6  import BookingSystem from "../pages/BookingSystem";
7  import UserAuthentication from "../pages/UserAuthentication";
8  import BookingHistory from "../pages/BookingHistory";
9  import AdminDashboard from "../pages/AdminDashboard";
10 import PaymentGateway from "../pages/PaymentGateway";
11 import ETicketGeneration from "../pages/ETicketGeneration";
12 import Notifications from "../pages/Notifications";
13 import FiltersSorting from "../pages/FiltersSorting";
14
15 import "../App.css";
16
17 function App() {
18   return (
19     <Routes>
20       <Route path="/" element={<Home />} />
21       <Route path="/flight-search" element={<FlightSearch />} />
22       <Route path="/seat-selection" element={<SeatSelection />} />
23       <Route path="/booking-system" element={<BookingSystem />} />
24       <Route path="/user-authentication" element={<UserAuthentication />} />
25       <Route path="/booking-history" element={<BookingHistory />} />
26       <Route path="/admin-dashboard" element={<AdminDashboard />} />
27       <Route path="/payment-gateway" element={<PaymentGateway />} />
28       <Route path="/e-ticket" element={<ETicketGeneration />} />
29       <Route path="/notifications" element={<Notifications />} />
30       <Route path="/filters-sorting" element={<FiltersSorting />} />
31     </Routes>
32   );
33 }
34
35 export default App;
```

Installation/setup instructions

The **Flight Search and Booking System** is a web-based application developed using **React.js** for the frontend, **Node.js** and **Express.js** for the backend, and **MySQL** as the database management system. Before running the application, the required software such as **Node.js**, **npm**, **MySQL Server**, **Visual Studio Code**, and a modern web browser should be installed on the system. Optionally, **Git** can be used to clone the project repository and manage version control.

After downloading or cloning the project, all necessary project dependencies are installed using the **npm install** command in both the frontend and backend directories. These dependencies include the libraries and packages required for building the user interface, handling server-side operations, managing routing, processing API requests, and connecting to the database. Once the installation is completed, a MySQL database is created, and the required tables are imported or configured according to the project requirements.

The backend is then configured by creating an **environment (.env) file**, where important configuration details such as the database host, database name, username, password, server port, JWT secret key, and API keys are stored securely. This approach prevents sensitive information from being hardcoded into the application and makes it easier to deploy the project in different environments. The backend server is started using Node.js, while the React frontend is launched separately. Once both servers are running successfully, the frontend communicates with the backend through RESTful APIs, enabling smooth interaction with the database and other application services.

Finally, the application can be accessed through a web browser using the local development URL. Users can register new accounts, log in securely, search for available flights, select seats, complete bookings, make online payments, and receive electronic tickets. Administrators can log in to the dashboard to manage flight schedules, users, and bookings. After verifying that all modules function correctly, the Flight Search and Booking System is considered successfully installed and ready for development, testing, or deployment.

User manual or guide

The **Flight Search and Booking System** is designed to provide users with a simple, secure, and convenient platform for searching and booking airline tickets online. To begin using the application, users must first register by providing basic details such as their name, email address, phone number, and password. After successful registration, users can log in using their registered email and password. Once authenticated, they are redirected to the home page, where they can access all the features of the system.

To search for flights, users need to enter the departure city, destination city, travel date, number of passengers, and preferred travel class. After submitting the search request, the system retrieves and displays a list of available flights, including airline names, departure and arrival times, travel duration, ticket prices, and seat availability. Users can further refine the search results using filters such as price range, airline, departure time, or number of stops to find the most suitable flight.

After selecting a preferred flight, users proceed to the booking section, where they enter passenger details such as name, age, gender, and contact information. The system verifies seat availability and displays the booking summary before directing the user to the payment page. Users can complete the payment using the available online payment methods, such as UPI, debit card, credit card, or net banking. Upon successful payment, the system confirms the booking, generates a unique Booking ID and PNR number, and displays an electronic ticket containing all travel details. A copy of the e-ticket can also be downloaded or sent to the user's registered email address.

Users can access the **Booking History** section to view all previous and upcoming reservations, check booking status, or cancel eligible bookings according to the applicable policies. They can also update their profile information and change their account password whenever required. Administrators have access to a separate dashboard where they can manage flights, monitor bookings, maintain user records, and update airline schedules. Overall, the Flight Search and Booking System provides an intuitive and user-friendly interface that enables users to complete the entire flight booking process efficiently while ensuring secure transactions and reliable access to travel information.

Geo Tag photos with guide



